## Zadání, inspirace a stručný opis programu:

Program FEM využíva metódu konečných prvkov (Finite Elements Method) pre numerické riešenie Poissonovej rovnice na užívateľom zadanej 2D polygonálnej oblasti.

Poissonova rovnica je parciálna diferenciálna rovnica druhého rádu. Zadanie úlohy pozostáva z pravej strany f(x,y), polygonálnej oblasti Ω, a počiatočných podmínek Neumannových a Dirichletových. Po spustení programu FEM je užívateľ vyzvaný na zvolenie súboru, ktorý obsahuje vstupné dáta pre úlohu, ktorú si užívateľ želá riešiť. Vstupný súbor má príponu .py, obsahuje python objekty bližšie špecifikované v nižších odsekoch.

Počas písania programu FEM ma som čerpal vedomosti a inspiraci z článku “Remarks around 50 lines of Matlab: short finite element implementation” [1].

Program FEM začne tým, že používateľom zadaný polygón premení na trojuholníkovú sieť (triangulaci) pomocou knižnice gmsh, následne z tejto siete vytiahne dáta o topológii a geometrii triangulace. For-cyklom prejde každý trojuholníkový element siete, na ktorom vykoná jednoduché výpočty a postupne zloží globálnu maticu tuhosti A a vektor pravej strany **b** - k tomuto postupu sa pristupuje preto, že obecne zložité “objemové” integrály, ktoré sa vyskytujú v takzvanej slabej formulaci Poissonovej rovnice sa stanú triviálnymi, keď ich počítame na trojuholníkovom elemente namiesto celej oblasti Ω. Následne program prejde všetky zadané Neumannove počiatočné podmienky a spracuje ich podobne. Dirichletove počiatočné podmienky sa využijú na to, aby program odseparoval tie uzly triangulace, na ktorých hodnoty funkce počítat nechceme - celá sústava A**x** = **b** ktorú program zložil má totiž presne toľko rovníc, kolik je uzlov v síti, ale my všetky počítať nemôžeme, keďže v nejakých je hodnota funkce vopřed dána. Zo sústavy teda odrežeme tie riadky a stĺpce, ktoré Dirichletove podmienky určujú, a následne hotovú sústavu v tvare A´**x** = **b**´ vyriešime. Vektor **x** získaný ako riešenie sústavy reprezentuje hodnoty hledané funkce (splňující poissonovu rovnici se zadanými podmínkami a pravou stranou) v uzlech vytvorených triangulací. Pomocou týchto hodnot také spočítáme hodnotu gradientu hledané funkce v každém trojúhelníkovém elementu. Po vyhlazení získaných dat na po částech lineární funkci u(x,y) program vyobrazí 2D heatmapu funkce u(x,y), 3D heatmapu so zjemneným rozlíšením, 2D vektorový graf gradientu funkce u.

## Datové struktury:

Všetky požiadavky užívateľa (súradnice rohov polygónu, okrajové podmienky, pravá strana, hustota siete) sú v slovníku (dict). Topológia a geometria vygenerovanej siete sú uložené v maticiach trojuholníkových elementov a súradníc (np.ndarray). Na mapovanie interných uzlových tagov knižnice gmsh na maticové indexy slúži prekladový slovník (dict[int, int]). Sústava rovníc je reprezentovaná riedkou maticou (sp.csr_matrix, pri zostavovaní sa používa DOK formát) a vektormi (np.ndarray).

## Popis programu:

Zbehnutie programu od kontroly vstupných dát až po vizualizáciu jeobsiahnuté vo funkcii start_fem. Interakcia podprogramov je čiste lineárna, teda výstupné dáta z jednej

funkcie vstupujú priamo ako argumenty do ďalšej. Hlavné výpočtové bloky sú rozdelené do podprogramov:

Funkcia validate_input sleduje správnosť vstupných dát. Kontroluje sa tu prítomnosť povinných kľúčov v slovníku na vstupe, správnosť dátových typov , dostatočný počet vrcholov polygónu, platnosť a existencia zadaných okrajových podmienok. Pre mnohé (dúfam, že takmer všetky) chyby je tu samostatný raise blok s hlásením konkrétne akej chyby sa užívateľ dopustil.

Funkcia generate_mesh berie na vstupe skontrolované dáta a vytvorí pomocou gmsh trojuholníkovú sieť pokrývajúcu zadanú polygonálnu oblasť Ω.

Funkcia extract_mesh_data vyťahuje dáta z vyhotovenej siete. Jej výstupom je matica súradníc všetkých uzlov siete, prekladový slovník (vnútorné gmsh indexovanie uzlov siete je netransparentné a teda nie dobre predvídateľné, používa sa teda prekladový slovník ktorý na základe tagu uzla povie, aký je jeho index v matici súradníc), matica elementov a počet uzlov v sieti.

Funkcia assemble_system prijíma topológiu a geometriu siete, a z užívateľských dát si vytiahne navyše funkciu pravej strany. Cyklom prechádza všetky elementy, počíta lokálne matice tuhosti v každom z nich pomocou metód odvodených v teórii metody konečných prvkov. Z toho poskladá celú globálnu maticu tuhosti A a vektor pravej strany **b**.

Funkcie apply_dirichlet a apply_neumann modifikujú sústavu na základe okrajových podmienok zadaných užívateľom. apply_neumann pričíta hodnoty vyplývajúce zo slabej formulace poissonovej rovnice do vektora pravej strany. apply_dirichlet zapisuje hodnoty predpísané užívateľom vrámci počiatočnej podmienky do vektora riešení a vracia zoznam uzlov s predpísanou hodnotou, čo je nutné pre neskoršiu redukciu dimenze systému, ako sme opísali vyššie.

Funkcia solve_system pomocou zoznamu Dirichletových uzlov (výstup z funkce apply_dirichlet, tak nazývame uzly v ktorých hodnotu máme od užívateľa predpísanú a teda nie sú “voľné”) odstráni zo sústavy rovnice a koeficienty tak, aby sústava, ktorá ostane obsahovala len rovnice pre voľné uzly. Zredukovanú sústavu vyrieši pomocou funkcie z knižnice scipy špecilizovanej na riešenie sústav Ax=b s riedkou maticou A a výsledky zapíše do globálneho vektora riešení, ktorý sa následne odovzdá vizualizačným funkciám.

## Vstupné dáta

Program FEM prijíma vstup z dokumentu s príponou .py. Tento dokument v sebe musí obsahovať nasledovné: a) slovník input_data so všetkými dátami potrebnými k správnemu zadaniu želanej úlohy; b) dostatočný počet definovaných funkcií dvoch priestorových premenných

**Ad a)**:
Užívateľ do .py súboru jednoducho definuje premennú typu dict s názvom input_data, a slovník vyplní nasledovne:

1. povinným kľúčom v slovníku je kľúč “nodes”, ku ktorému je potrebné priradiť maticu súradníc rohov želaného polygónu. Matica je v typickom tvare, jej riadky tvoria dvojice čísel (int/float), teda typicky môže matica vyzerať nejak takto: [[0,0],[1,1],[2,3]]. Podmienkou je, aby počet riadkov matice bol aspoň 3, aby v každom riadku boli zadané presne dve číselné súradnice a aby boli tieto vrcholy v správnom poradí. Správnym poradí vrcholov sa myslí to, že program bude zadané vrcholy čítať postupne rad po rade - vždy teda jeden vrchol prečíta, pridá do existujúceho modelu a rovnou čiarou ho spojí s predošlým prečítaným bodom. Aby takto čítaný vstup nakoneic dal valídnu polygonálnu oblasť, nesmú sa kreslené čiary spájajúce dva vrcholy nikdy pretnúť (iba v bodoch vrcholov samotných). Upozorňujeme, že chybné poradie bodov je jeden z mála chybných vstupov, ktorý program sám nedetekuje a užívateľa naň explicitne neupozorní ešte pred zbehnutím výpočetního jadra, ako to býva u iných chybných vstupov. Príkladom zle zadaného polygónu striktne kvôli poradiu bodov by mohol byť: [ [0,0],[1,1],[0,1],[1,0] ].
2. ďalším povinným kľúčom je “dirichlet”, ku ktorému sa priraďuje matica, ktorej riadky tvoria dvojice typu [int/float, Callable]. Prvý prvok označuje poradie hrany, na ktorej danú dirichletovu počiatočnú podmienku zadávame - číslovanie hrán je intuitívne a nadväzuje na predošlú pasáž, totiž hrany sú číslované tak, že hrana číslo 1 spája vrchol zadaný ako prvý s vrcholom zadaným ako druhým, a obecne hrana s číslom n spája vrcholy n a (n+1), až na poslednú hranu, ktorá očividne spája posledný zadaný bod s prvým. Druhý prvok - Callable - označuje funkciu, ktorou na zadanej hrane chceme počiatočnú podmienku definovať. Môže ísť o ľubovolnú reálnu funkciu dvoch premenných, viacej v časti b) tejto kapitoly. Hodnota priradená tomuto kľúču môže vyzerať napríklad takto: [[2,funkce2],[12,funkceFunkce]]
3. Nepovinným kľúčom je “neumann”, čo je ekvivalent počiatočnej podmienky predstavenej v odseku 2, ale z pohľadu matematickej analýzy nezadávame na danej hrane hodnotu hľadanej funkcie, ale hodnotu jej derivace. Z teórie parciálnych diferenciálnych rovníc víme, ža takáto podmínka není nutná k tomu, aby úloha bola jednoznačne řešitelná, preto nie je uvedená ako povinný klíč v slovníku. Štruktúra zadávania je úplne zhodná s odsekom 2.
4. povinným kľúčom je “right_side”, ku ktorému je v slovníku nutné priradiť Callable objekt - v našom prípade reálnu funkciu 2 premenných. Toto je funkce, ktorá v zadaní poissonovej rovnice stojí na pravej strane, teda pre hľadanú funkci u(x,y) platí laplace(u) = f in Ω.
5. nepovinným kľúčom je “mesh_size”, ku ktorému sa priraďuje číslo (float/int, aj keď int je väčšinou nepoužiteľná voľba), ktoré manuálne nastavuje hustotu siete generovanej v prvom kroku výpočtu. Defaultná hodnota je 0.1, no pri veľkých polygónoch sa môže oplatiť znížiť pre zrýchlenie výpočtu. (nižšie číslo znamená hustejšiu sieť)

**Ad b)**:
Ako je z odsekov 2,3,4 jasné, vstupný dokument potrebuje dostatočný počet definovaných Callable objekt, teda reálnych funkcií dvoch premenných. Funkcie sa definujú typicky, teda napríklad takto:

```
import math
def funkceFunkce(x,y):
    return math.sin(x)
```

V zadávání počátečních podmínek v slovníku sa potom používa iba meno funkcie - v našom prípade funkceFunkce.

## Výstupné dáta:

Po zbehnutí výpočtu je užívateľ prezentovaný tromi grafmi, ktoré sme mierne opísali v úvode dokumentu. V poradí to sú:
**Figure 1**: 2D mapa zadanej polygonálnej oblasti, na ktorej ju zvýraznená (čiernou mriežkou) triangulace, na které výpočet probíhal. Vedľa grafu je farebná legenda, nakoľko ide o heatmapu. Vo volnejšom slova zmysle je táto heatmapa “presná”, nakoľko farebná schéma aplikovaná na jednotlivých trojuholníkoch je robená tak, že na ploche trojuholníka sa farba mení “lineárne”.
**Figure 2**: Vektorový graf ukazujúci intuitívne porozumenú veličinu toku nájdenej funkcie U(x,y). Veľkosť šípky ukazuje magnitúdu veličiny.
**Figure 3**: 3D plot so zjemneným rozlíšením. K zjemneniu dochádza až po samotnom výpočte, teda je to “zadarmo” vďaka metóde refine_field v knižnici matplotlib a občas vedie k lepšiemu zobrazeniu 3D grafu.

## Příklady:

**Příklad 1:** Máme veľký polygón (súradnice v řádu 10**3), s dvomi dirichletovymi a dvomi neumannovymi počiatočnými podmienkami. V tomto príklade sme nútený manuálne nastaviť parameter mesh_size - s hodnotou mesh_size v řádu 10 trvá výpočet pár minút, s hodnotou mesh_size = 150 trvá asi sekundu. Přesný vstupný kód:

```
def prava_strana_velky(x, y):

      return 0.0

def diri_spodok(x, y):

      return 0.0

def diri_vrch(x, y):

      return 150.0

def neu_prava_strana(x, y):

      return 25.0

input_data = {

"nodes": [ [0.0, 0.0], [2000.0, 0.0], [3000.0, 1000.0], [3000.0, 4000.0],
[2000.0, 5000.0], [1000.0, 5000.0], [0.0, 4000.0], [1000.0, 2000.0] ],

"dirichlet": [ [1, diri_spodok], [5, diri_vrch] ],

"neumann": [ [3, neu_prava_strana], [4, neu_prava_strana] ],
```

```
"right_side": prava_strana_velky,

"mesh_size": 0.5

}
```

**Příklad 2:** Tenká trubica s pevnou dirichletovou podmienkou na pravej hrane a na lavej hrane, s nulovou neumannovou podmienkou na hornej a dolnej hrane.Parameter mesh_size nechávame defaultný.

```
def prava_strana_trubica(x, y):

      return 0.0

def diri_teply_kraj(x, y):

      return 100.0

def diri_studeny_kraj(x, y):

      return 20.0

def neumann_izolacia(x, y):

      return 0.0

input_data = {

"nodes": [[0.0, 0.0], [10.0, 0.0], [10.0, 0.5], [0.0, 0.5]],

"dirichlet": [ [4, diri_teply_kraj], [2, diri_studeny_kraj] ],

"neumann": [ [1, neumann_izolacia], [3, neumann_izolacia] ],

"right_side": prava_strana_trubica }
```

### Citace:

[1]: Alberty, J., Carstensen, C. & Funken, S.A. Remarks around 50 lines of Matlab: short finite element implementation. *Numerical Algorithms* **20**, 117–137 (1999). https://doi.org/10.1023/A:1019155918070